INTRODUCTION TO DIGITAL IMAGE PROCESSING

Final Project Report

Written by:

Itzik Nsimov

Iris Eting

Etai Wigman

Nadav Orenhstein





Contents

Contents 2
Introduction 3
Idea 3
Description 3
Motions 4
System Structure 6
Challenges 8
Assumptions and Limitations 9
Algorithms 10
Background Subtraction 10
Threshold 10
Gaussian filter 10
Median filter 11
Canny edge detection 11
Hough transform 12
HSV Color Domain and Color Analysis 12
Find Contours 13
Solutions 14
Midpoint Outcomes 20
Analysis 25
Lean Detection 25
Conclusion 29
Summary 31
Bibliography 32
Links 32

Introduction

Idea

In our computer vision and image processing-driven interactive game interface development, our primary focus revolved around immersing players in a lifelike snowboarding simulation. We aimed not just to let players control an avatar but to transport them into the game environment, where they could physically experience navigating mountain slopes on a virtual snowboard. Our endeavor was to replicate the precise physics of snowboarding, ensuring that every turn, jump, and lean on the virtual board mimicked real-world dynamics. This meticulous attention to physical accuracy was crucial in delivering an intuitive gameplay experience, where players' actions seamlessly translated into the game with unparalleled realism. In achieving this goal, we leveraged advanced image processing tools to analyze real-time player movements and translate them into responsive actions within the virtual environment. This integration of cutting-edge image processing techniques played a pivotal role in ensuring the seamless fusion of physical realism and interactive gameplay.

Description

We developed a user interactive platform aimed at augmenting the gaming experience within the existing 'Snowboard Party' game for Windows PC. Our innovation centered on seamlessly translating real-time player movements into in-game actions. Utilizing live video streams of the player, we employed sophisticated image processing tools to accurately discern the validity of each potential motion. Upon confirmation, the system digitally triggered the corresponding actions within the game by simulating key presses on the keyboard. This intricate integration of image processing techniques with game

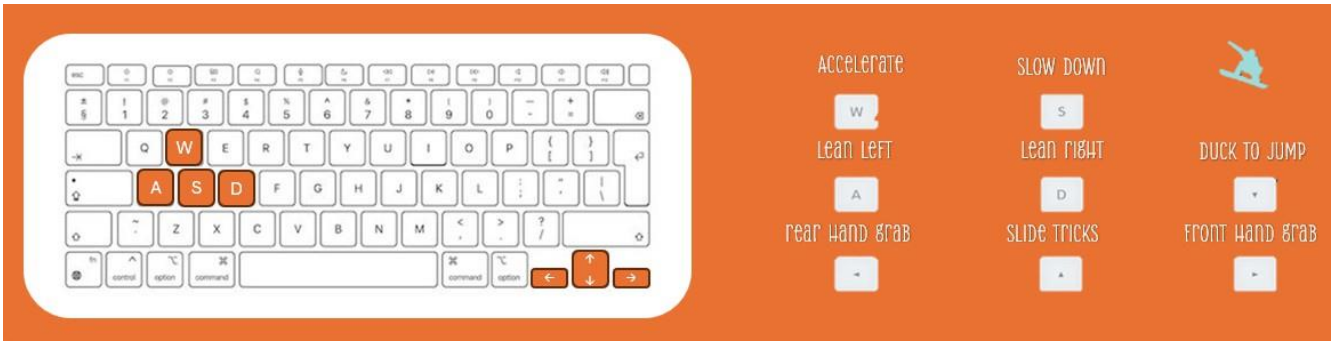


Figure 1-game controls

mechanics ensured a dynamic and immersive gameplay experience, where player actions directly influenced on-screen outcomes.

The possible player motions and key press are:

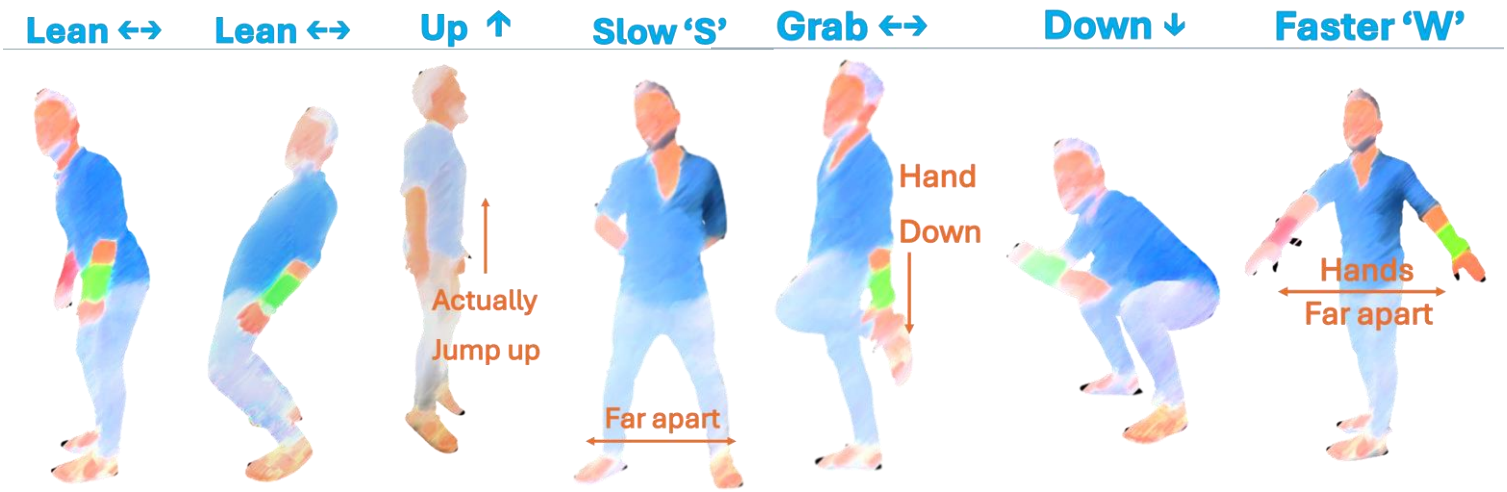


Figure 2-motions

possible complications can be signaling "Combo" actions to the game platform. Further game instructions can be found under 'Links'.

Motions

❖ Lean



To execute turning left and right in the game, players simply shift their balance towards the desired direction, mimicking the natural movement required on a physical snowboard.

❖ squat



In order to crouch down and get ready for a jump the player simply squats down in place and releases this stance to jump.

❖ Jump



In our snowboarding game, players physically jump in real life to execute jumps within the game, enhancing the immersive experience and adding a physical element to gameplay. The jump key allows you to perform slide tricks and more.

❖ Slow



To slow down or halt within the game, players adopt a wide stance, intuitively mirroring real-life snowboarding. This stance is characterized by a visible triangle formed between the player's legs, ideally with a top angle ranging from 80 to 120 degrees.

❖ Grab



Similar to real-life stunts, where surfers enhance angular velocity by grabbing their snowboard mid-air, our game enables players to perform these maneuvers by reaching towards the ground or their feet, indicating for a grab.

❖ Fast



To accelerate in the game, players orient their upper body forward and extend both hands outward and forward, maintaining them at about the same height.

System Structure

We opted to position the camera two meters away from the player to ensure that the entire body is captured within the frame. The system's input consists of frames sampled by the camera. We've developed a custom module to handle these frames, managing camera IO and buffering results. This setup envelops the image stream, ensuring a seamless real-time experience while enabling parallel image processing computations.

At the core of the system lie the image processing blocks. Firstly, we extract a binary mask using denoising filters and outlier removal methods, custom-tailored to isolate and segment the player's body within a region of interest representation. This binary mask serves as the foundation for subsequent analyses.

On one end of the system, a "controller" module interfaces directly with the operating system to simulate keyboard inputs, mirroring human keystroke actions in terms of type, duration, and frequency.

Between the input and output modules are inference blocks, each receiving specific input data to determine necessary actions:

1. The output of the Hough lines transform, applied to edges detected in the binary mask using Canny edge detection, is processed by the first inference block to detect "slow down" action initiated by the user.
2. The binary mask is utilized to estimate the player's center of mass, segment their lower and upper body, and determine their angle of lean. This data is further processed by the lean inference block, while the proximity and locations of the lower and upper centers of mass are inputted into the duck/jump inference block.
3. By examining changes in center of mass locations along the horizontal axis, we estimate the vertical velocity. Averaging this over a time window provides the input for the jump/slide inference block, which assesses whether the player has jumped in the air.
4. Leveraging the region of interest generated during binary mask creation, we focus on the specific area of the frame containing the player. We then

perform a color space conversion to HSV to detect the left and right "gloves", representing the player's hands. These locations are fed into the final inference block, determining whether a grab was attempted and by which hand, and whether “fast” was signaled.

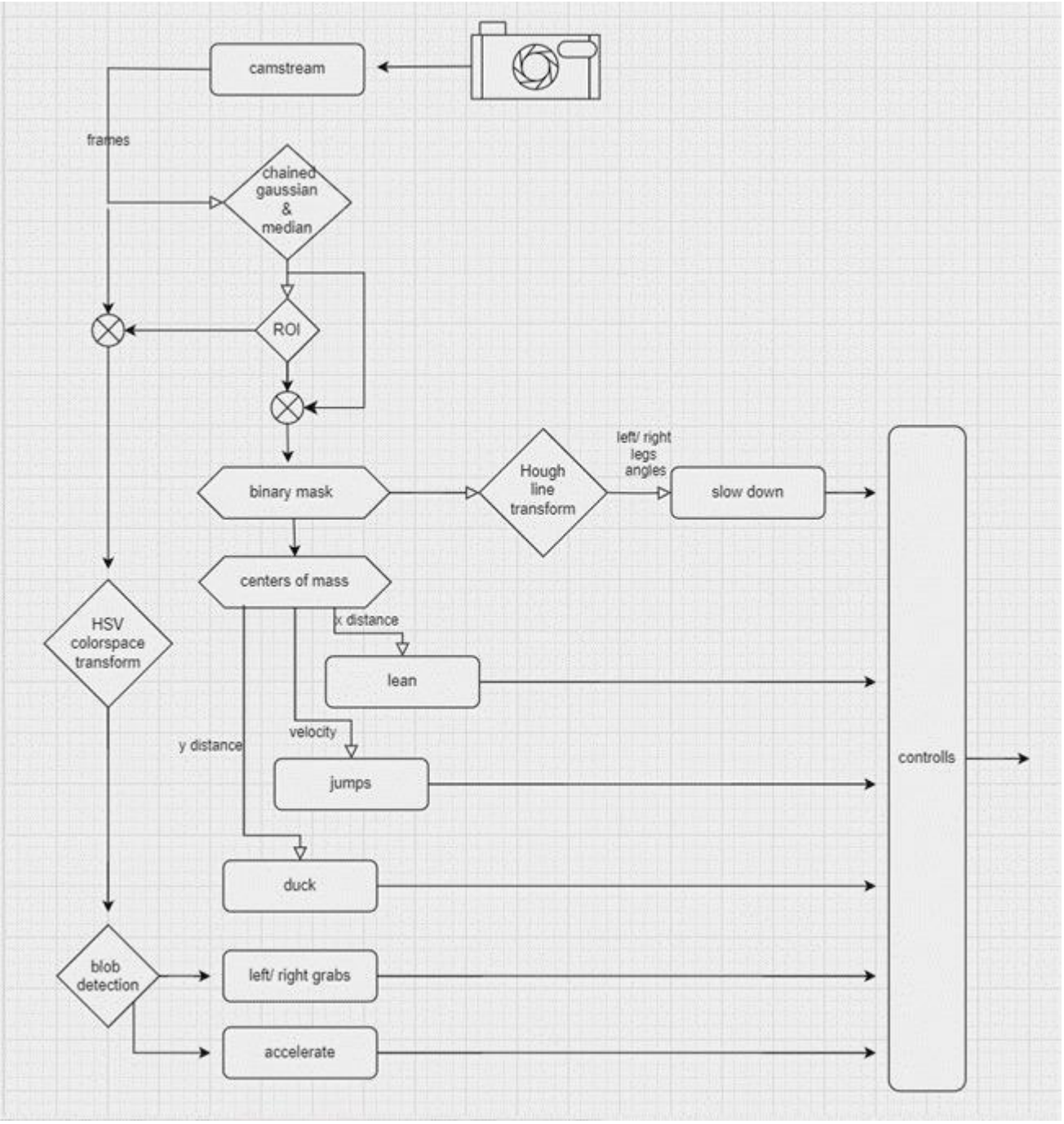


Figure 3-system flow chart

Challenges

During the project we encountered various challenges that we will note here, the solutions to these challenges will be explained later:

- **Creating denoised binary mask** - environmental noise, light flickering, exposure fluctuations and minor background movement needed to be accounted for.
- **Finding the center of Masses** - Deciding the centers of mass for the overall, upper, and lower body by determining which pixels will be taken into consideration and which will be seen as outliers.
- **Recognizing the red and green marks** - Ensuring that the marks that were found were not background interference as well as deciding the color spectrum for the marks.
- **Recognizing the wide stance position -** Accurately detecting the wide stance using the triangular shape while avoiding false negatives/positives.
- **Balancing thresholds -** Deciding the thresholds for each action to minimize false negatives/positives while retaining the fluidity and responsiveness of the gameplay. Additionally, it's crucial to differentiate between similar actions to ensure accurate gameplay.
- **Tailoring Thresholds to an unknown user -** Each user has a different body shape and size, as well as movements.
- **Accurate game's interface control -** Defining the way we perform the actions in game. Achieving precise control over the game's interface posed a significant challenge, requiring a solution that could manage prolonged key presses without interrupting other processes.
- **Live processing -** In this real-time game, we addressed latency and gameplay smoothness to ensure optimal performance, recognizing the critical importance of every fraction of a second.

Assumptions and Limitations

During our project we used a few assumptions and limitations:

- Constant background - we assumed a static background for the creation of the binary mask, this assumption is more flexible as our code was robust enough to counteract minor noises.
  As for large noises, such as permanent lighting change or moving the camera, we implemented a quick solution for rescanning the background without the need to restart the system, simply by pressing ‘p’ to pause the game and ‘L’ to rescan the background.

- Distance - The player needs to be far enough for the camera to capture the whole body, if some of the body parts of the user will be cut off, some of the actions will be impacted by it, the user will still be able to play the game, but the accuracy will be damaged.

- Accessories - We chose arm wraps over hand recognition, leveraging the HSV color domain to enhance functionality and calculation speed, while allowing the ability to create a more versatile array of actions using those accessories.

Algorithms

In this section we will run over the main algorithms we used in our project and give an overview explanation about them.

Background Subtraction

Background subtraction is a fundamental concept in image processing with versatile applications in object tracking, surveillance, and motion detection. It involves separating foreground objects from a static or dynamic background within an image or video sequence. This process facilitates the extraction of relevant information while filtering out background noise, enhancing the accuracy of scene analysis. Typically, background subtraction algorithms compare pixel intensity or color values in consecutive frames to a reference background model. Deviations from this model signify foreground objects, while consistent pixels are attributed to the background.

Threshold

Thresholding is a fundamental technique in image processing used to create binary masks by separating pixels into two categories based on their intensity values. The process involves selecting a threshold value and classifying pixels as either foreground or background depending on whether their intensity is above or below the threshold. This method is crucial for tasks like object segmentation.

Gaussian filter

The Gaussian filter is a type of a low-pass filter for smoothing and noise reduction. It operates by convolving an image with a Gaussian kernel, which assigns weights to surrounding pixels based on their distance from the center pixel. This weighting scheme ensures that nearby pixels contribute more significantly to the smoothed output than distant ones, resulting in a blur effect that helps eliminate high-frequency noise while preserving important image features. The Gaussian filter is particularly effective in removing Gaussian noise, which is characterized by a bell-shaped distribution similar to the Gaussian kernel. By adjusting the size of the Gaussian kernel, users can

control the amount of smoothing applied to the image, striking a balance between noise reduction and preservation of image details.

Median filter

The median filter is a popular technique in image processing used for removing noise while preserving edges and fine details. Unlike traditional linear filters, such as the Gaussian filter, which compute the weighted average of neighboring pixel values, the median filter replaces each pixel's value with the median value of its surrounding neighborhood. This approach is particularly effective at removing impulse noise, also known as salt-and-pepper noise, which randomly corrupts individual pixel values with extreme values. By taking the median value, the filter effectively ignores outliers caused by noise while preserving the structure of the underlying image. The size of the neighborhood window determines the extent of noise reduction, with larger windows providing greater smoothing at the cost of potential blurring of edges.

Canny edge detection

The Canny edge detection algorithm is a fundamental method employed in digital image processing to identify edges.

The CV implementation of canny edge detection first applies a Gaussian filter to the input image, aiming to mitigate noise and ensure accurate edge detection. Subsequently, it computes the gradient magnitude and direction at each pixel, discerning regions of rapid intensity variation. To refine the detected edges further, performing non-maximum suppression to thin the detected edges to single-pixel width. Following this, the algorithm employs hysteresis thresholding, involving the definition of two thresholds: a high threshold and a low threshold. Pixels exhibiting gradient magnitudes surpassing the high threshold are classified as strong edges, whereas those falling below the low threshold are deemed non-edges. Edges with gradient magnitudes lying between the two thresholds are categorized as weak edges and are connected to strong edges to form coherent edge contours. In essence, the Canny edge detection algorithm represents a sophisticated and versatile tool, pivotal in tasks such as image segmentation, object detection,

and feature extraction, owing to its ability to accurately extract edge information from images.

Hough transform

The Hough transform is a computational technique employed in computer vision to identify basic geometric forms within images through the conversion of image data into a parameter space. For instance, in detecting straight lines, each edge pixel in the image contributes to a set of potential lines that might intersect it, delineated in parameter space by their slope and intercept. Subsequently, an accumulator array is generated by the Hough transform, where each element corresponds to specific slopes and intercepts. Local maxima in this array denote the most probable lines in the image. The parameters utilized to represent these lines in the accumulator array are ρ and
θ, symbolizing the distance from the origin to the line along a perpendicular and the angle between the line and the x-axis, respectively. Renowned for its efficacy in identifying simple geometric forms within images, the Hough transform serves as a cornerstone in numerous computer vision applications.

HSV Color Domain and Color Analysis

The HSV (Hue, Saturation, Value) color domain is a widely used color space in image processing for identifying specific colored objects within images. Unlike the RGB color space, which represents colors as combinations of red, green, and blue values, HSV separates color information into three components: hue, saturation, and value. Hue represents the color itself, such as red, green, or blue, while saturation measures the intensity or purity of the color, and value represents the brightness of the color. This separation of color components makes the HSV color space particularly useful for identifying certain colored objects, as it allows for more intuitive manipulation of color information. By thresholding the hue component, for example, specific ranges of colors can be isolated, enabling the detection of objects with predefined color characteristics. Overall, the HSV color domain is a valuable tool in image processing for tasks such as object detection, segmentation, and tracking based on color properties.

Find Contours

The "Find Contours" algorithm in OpenCV is a fundamental technique used to detect and extract the contours of objects within a binary image. This algorithm traces along the edges of white pixels, representing objects, using a modified Moore-Neighbor algorithm until it forms closed loops. These loops are saved as contours, which are then approximated to simplify their shape while retaining key features. The function outputs a list of contours, with each contour represented by an array of (x, y) coordinates. These contours can be utilized for various purposes such as object detection, segmentation, and area calculation, providing valuable information for image processing tasks.

Solutions

❖ Binary Mask

To facilitate essential computations in our algorithm, we begin by segmenting the image to generate a binary mask that isolates the player. By doing so, we track the player's position and posture in each frame.

Achieving denoised binary mask

The methodology outlined below encompasses a series of steps meticulously designed to achieve a denoised binary mask accurately representing the player's presence:

1. **Absolute Difference:** Initially, we compute the absolute difference between the current frame and the background frame, effectively highlighting areas of significant change indicative of the player's presence.
2. **Conversion to Grayscale:** Subsequently, we convert the difference image to grayscale, focusing solely on intensity values to represent the magnitude of the difference.
3. **Gaussian Filter:** Applying a Gaussian blur to the grayscale difference image serves to smoothen it by reducing high-frequency noise and irregularities, thus enhancing image quality.
4. **Median Filter:** Further refinement is achieved through the application of a median blur, which effectively eliminates salt-and-pepper noise, a common issue post-Gaussian filtering.
5. **Thresholding:** The smoothed image undergoes thresholding to create a binary mask, segregating foreground (player) from background. By selecting an appropriate threshold, we obtain a clear binary mask, effectively isolating the player.
6. **Region of Interest (ROI):** Post-binary mask generation, we focus on a specific region of interest containing the player. By calculating pixel distances from the center of mass within the binary mask, we filter out outliers based on a threshold-scaled standard deviation. The resulting

main mass pixels facilitate the determination of bounding box dimensions, effectively refining the segmentation process. Moreover, conserving computational resources and focusing analytical efforts on denoising the binary mask further.

These steps collectively contribute to the creation of a precise, denoised binary mask, crucial for accurate player segmentation in real-time gaming scenarios.

Finding the center of masses

As discussed in the algorithm section, our center of mass calculation involves averaging player pixel locations. To refine this process, we analyze the standard deviation ratio of distances from each pixel to the center of mass, setting a threshold to filter outliers. We ensure balance in the computed center of masses to mitigate skewness. These adjustments optimize accuracy and reliability.

Upper/Lower body separation

Given the center of mass of the binary mask, we then bisect it to create a lower and upper part, which in turn we find the center of mass on those parts, this is critical for determining the player's orientation and extracting features such as angle of tilt and more.

❖ Recognizing the red and green marks

We sought to identify our red and green accessories by transitioning to the HSV color domain for improved hue range representation. Focusing solely on the region defined by the binary mask, we generated a mask for the specified color range and identified contours within it. Iterating over these contours, we calculated their areas and checked if they exceeded a minimum threshold. If a contour surpassed this threshold and the current maximum area, we updated the center of contour mass and size accordingly. We used the color range update menu described below to perfect the color ranges. For visual representations, refer to the 'Midpoint outcomes' section.

❖ Recognizing the wide stance position

The wide stance could be recognized by the triangular shape formed by the player's legs. We utilized the Canny edge detector on the lower portion of the binary mask to extract edge information. Subsequently, the Hough transform was applied to detect straight lines from these edges. By employing angle selection criteria (angles between 20-40 degrees for the right leg and 140-160 degrees for the left leg), we identified and labeled potential left and right leg lines indicative of the wide stance position. These identified lines were then overlaid onto a blank frame. Upon detecting an intersection in the region of interest between a left and right leg line, execution of the 'Slow' action was signaled. For a more comprehensive understanding and visual representation, detailed explanations and demonstrations are provided within the 'Midpoint outcomes' section.

❖ Balancing thresholds

Optimizing thresholds is pivotal for precise motion detection, mitigating errors, and elevating user engagement. By fine-tuning these thresholds, we ensure accurate interpretation of movements, minimizing discrepancies and ensuring appropriate system response to user actions, thereby fostering a seamless user experience.

For example, in the 'get_player_position' function, thresholds are tailored to filter outliers, striking a balance between false positives and negatives, enhancing accuracy while minimizing noise. Similarly, in functions such as 'player_jump' and 'player_squat', thresholds are meticulously calibrated to accurately discern specific movements while reducing the likelihood of incorrect detections.

Tailoring thresholds to an unknown user involved a meticulous process of fine-tuning parameters to accommodate players of varying shapes and sizes. We conducted extensive testing with a diverse pool of users, collecting data on their movements and interactions with the system. By analyzing this data, we iteratively adjusted the thresholds to achieve optimal performance for all users, ensuring accurate detection and response to their actions. These final

thresholds were carefully calibrated to provide a seamless and tailored experience regardless of the user's physical attributes or gameplay style.

The following thresholds were achieved and optimized by hours of user experience and feedback:

```
self.squati = 500  # Smaller -> more squats easy, larger -> less squats, range [600,700] 650
self.leani = 13   # Smaller -> more action easily, larger -> less actions
self.grabi = 600   # Smaller -> LESS action easily, larger -> MORE actions
self.jumpi = 500   # Smaller -> more action easily, larger -> less actions
self.outlier_std_threshold = 5
```

Our optimal thresholds

Through strategic threshold adjustments to manage false positives and negatives, the project attains refined control and responsiveness, culminating in heightened user satisfaction and interaction.

- **Adjustability for User and scene**

  Edit Menus

We implemented an edit menu, accessed by pressing 'p' for pause, allowing customization of the algorithm to adapt to the current scene and environmental changes without program restart.

- **Color Range ‘c’**

We transitioned to the HSV color domain for enhanced hue range representation, with specific ranges for green and red colors. In case of difficulties recognizing red or green due to changes in light or scene affecting HSV values, users can access the color edit menu. Here, they can sample colors directly from the video frame by clicking on it, and then adjust the color range settings based on real-time observations.

- **Thresholds ‘b’**

We introduced thresholds edit menu, offering users the ability to adjust thresholds for each motion/action, influencing the sensitivity and likelihood of action activation. Lower thresholds increase sensitivity, making it easier for actions to be detected, while higher thresholds require more precise movements to trigger actions. Users can fine-tune these thresholds based on

their preferences and the specific requirements of their gaming experience. This feature ensures customization and optimization of the gameplay experience according to individual preferences and gameplay styles.

◇ Update Background ‘L’

For large noises, such as permanent lighting change or moving the camera, we implemented a quick solution for rescanning the background without the need to restart the system, simply by pressing ‘p’ to pause the game and ‘L’ to rescan the background.

◇ Return ‘r’

Simply return from pause and edit menus by pressing ‘r’.

◇ Height Sampling

Given that most players share a similar height range, we've optimized the default setting to accommodate the majority. Additionally, we've devised a system to calculate the user's height based on the upper and lower limits provided by the binary mask. Furthermore, players have the option to measure their height at the start of the game, ensuring a personalized experience.

❖ Enhanced Game Interface Control

To achieve precise control over the game's interface, we employed a Python library tailored for simulating keystrokes. Moreover, we developed a custom module specifically designed to meet our requirements. This module, operating on a separate thread, effectively manages prolonged key presses, controlling their duration without blocking other processes. Additionally, we implemented a custom camera stream module operating on a separate thread. This module continuously samples the camera and stores frames in a buffer. When requested, it retrieves the latest frame from the buffer, ensuring real-time processing without delays. All the while maintaining responsiveness and preventing interference with other processes.

❖ Real-Time Processing Optimization

We prioritized seamless real-time processing of camera streams within the game by implementing strategies to minimize IO wait times during webcam

sampling. This ensured that players experienced smooth and responsive gameplay without delays or interruptions. We opted for simple algorithms to ensure quick computation times, enabling real-time responsiveness.

Midpoint Outcomes

In our iterative development and debugging process, we implemented mid-level outcome displays to gain insights into our progress and identify discrepancies from our desired state. Utilizing a 4-frame output grid, we compared input and output frames along with various aspects of our functions. The two frames positioned on the right are intended to accompany the game interface for the player's view, while the two frames on the left serve for internal analysis. Additionally, the bottom-right frame provides visual feedback on the motions identified by the system as signals from the player to the game.

The 'Slow' position, correctly identified:

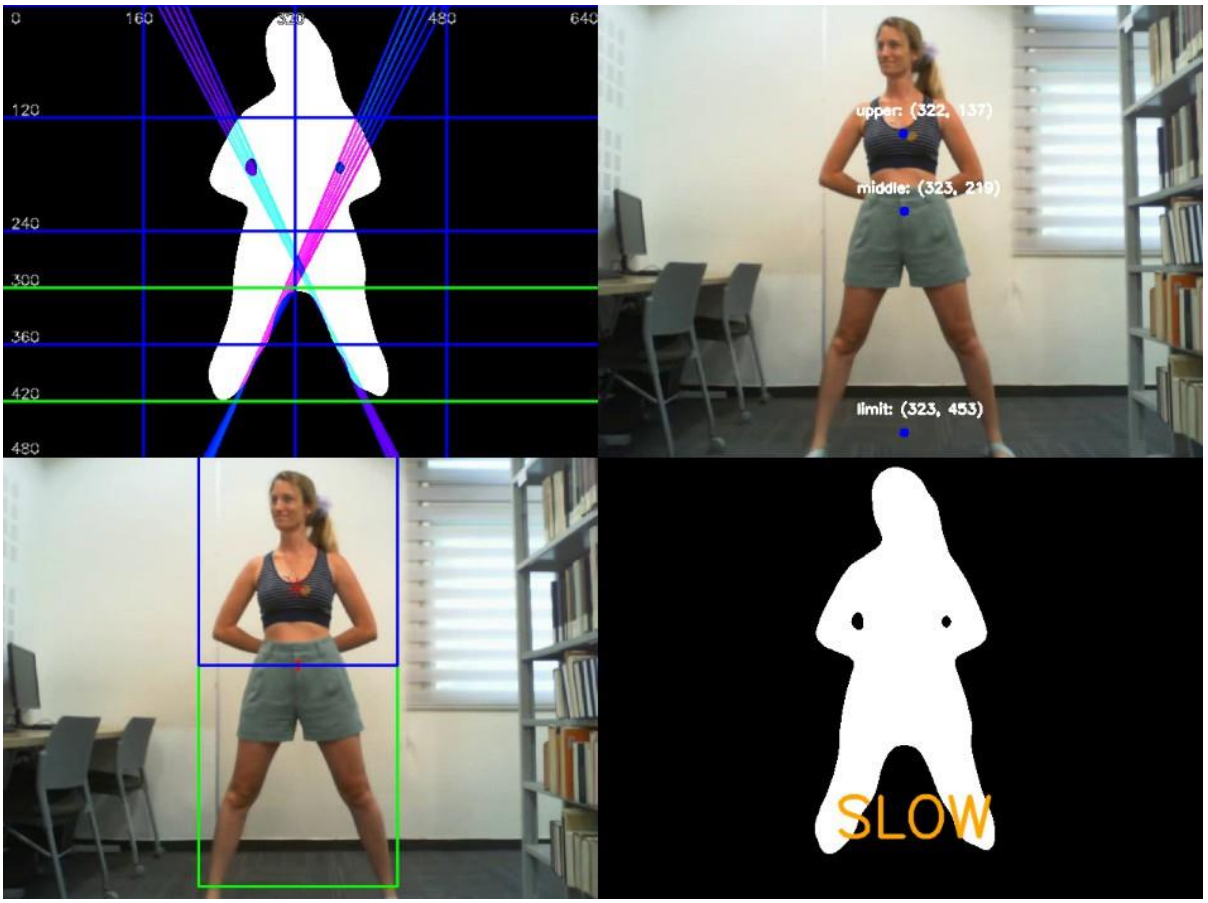


Figure 4-feedback

Possible False positives for the ‘Slow’ position have arisen from other potential 'triangles' detected following the Hough transform:

On the left: Undesirable intersections caused by lines that can be filtered out through angle selection criteria. Specifically, angles between 20-40 degrees for the right leg and 140-160 degrees for the left leg.

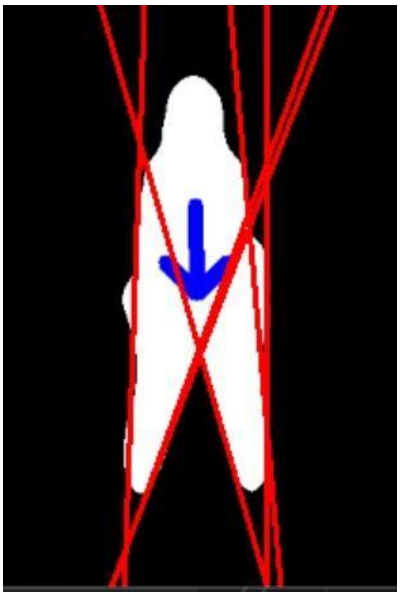


Figure 6-hough without angle criteria

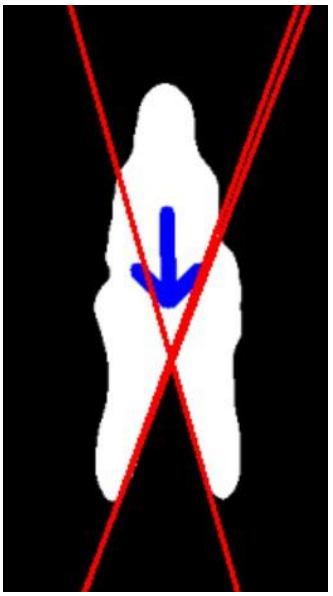


Figure 5-hough with angle criteria

On the left: Undesired intersections produced by the upper left outline (highlighted in green). To mitigate this issue, we excluded the upper half of the binary mask during this calculation.

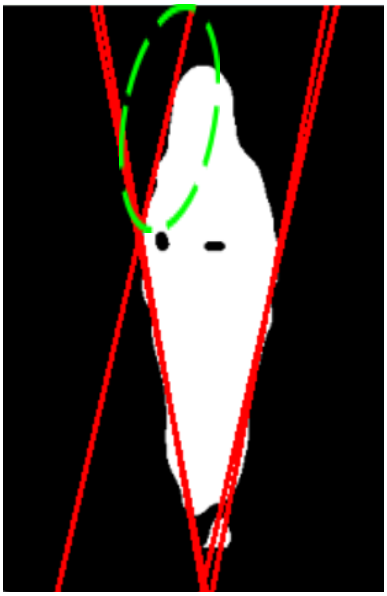


Figure 8-hough on all binary mask

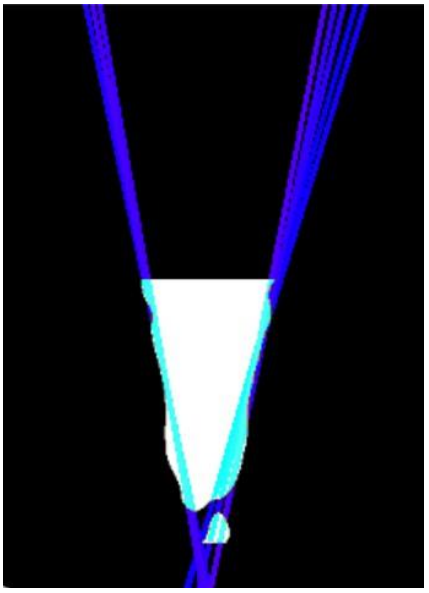


Figure 7-hough only on lower binary mask

We encountered a challenge with intersections occurring between lines generated for the same leg, where two lines within the degree range of 20-40 intersected within the area of interest. To address this, we opted to differentiate between lines representing the 'right leg' and 'left leg' by assigning distinct color channels. Specifically, each line associated with the 'right leg' was colored as (255, 10, 0), while each line linked to the 'left leg' was colored as (255, 0, 10). This allowed us to clearly see all the lines based on the 'blue=255' value and identify intersections between opposite legs by the presence of both 'green>=10' and 'red>=10'.

Another instance of a false positive for the 'Slow' position is shown below. In this case, two solutions have not yet been implemented: the angle selection criteria and the definition of the area of interest (set to 240 <= y <= 300 for the intersection).

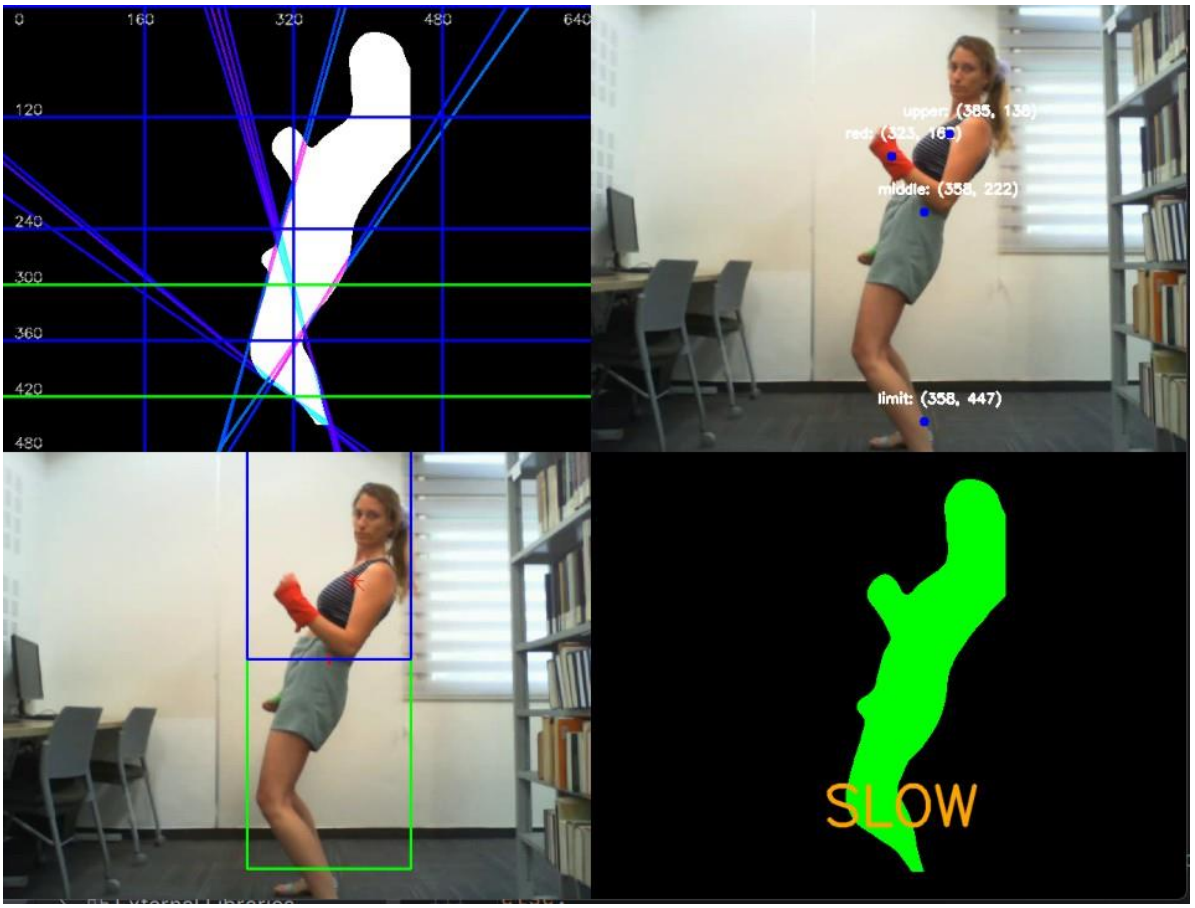


Figure 9-false identification of "slow"

The improved solution result can be seen below. As can be seen, the false positive has been removed. The remaining intersection that can be seen is outside of the area of interest.

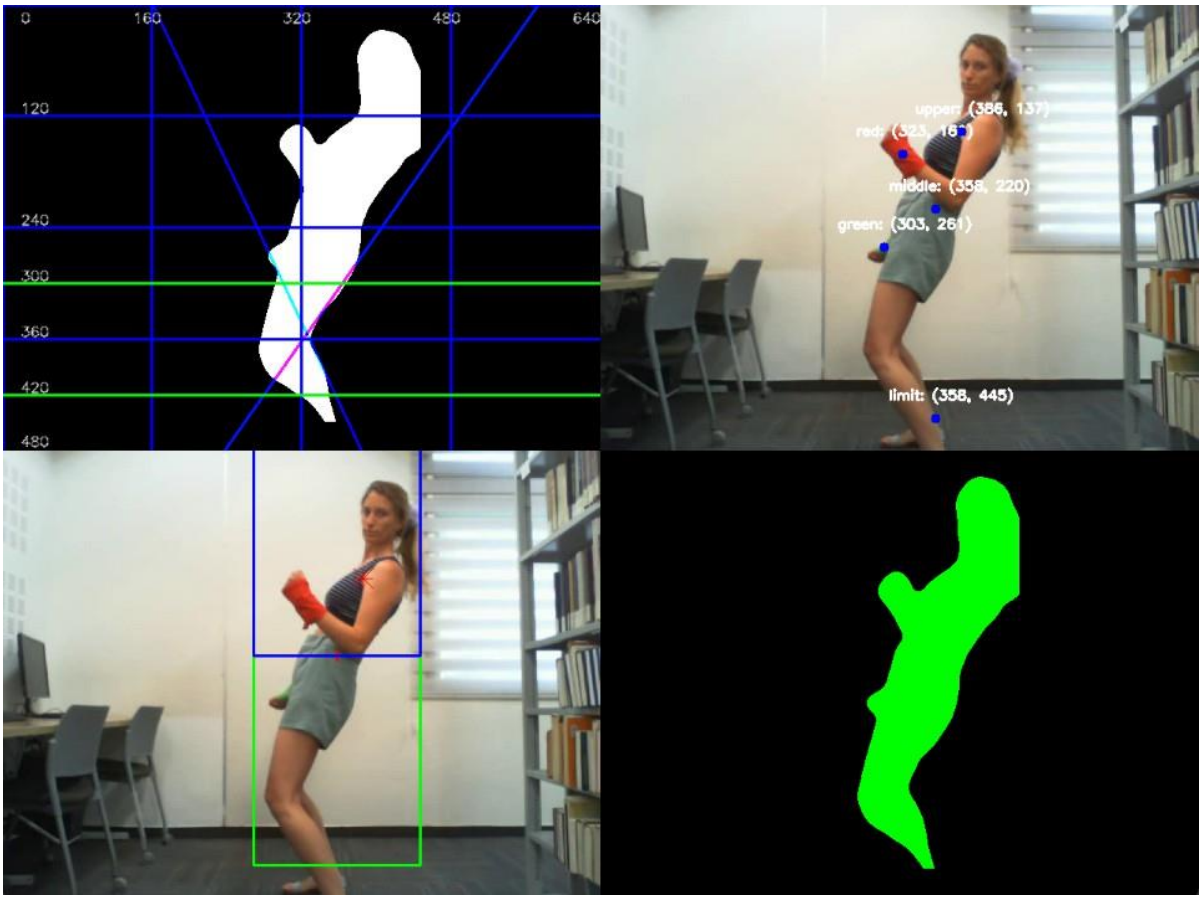


Figure 10-correct identification without "slow"

As demonstrated on the right, the 'Slow' motion is not activated because only one leg aligns with the angle selection criteria.

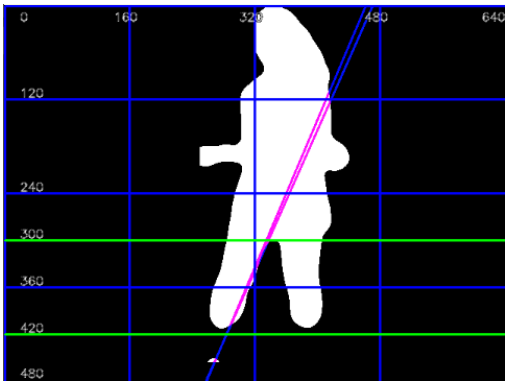


Figure 11-hough grid demonstration

Green and Red Accessories

Identification of green and red accessories was often inaccurate due to the presence of red and green objects in the background. For example:

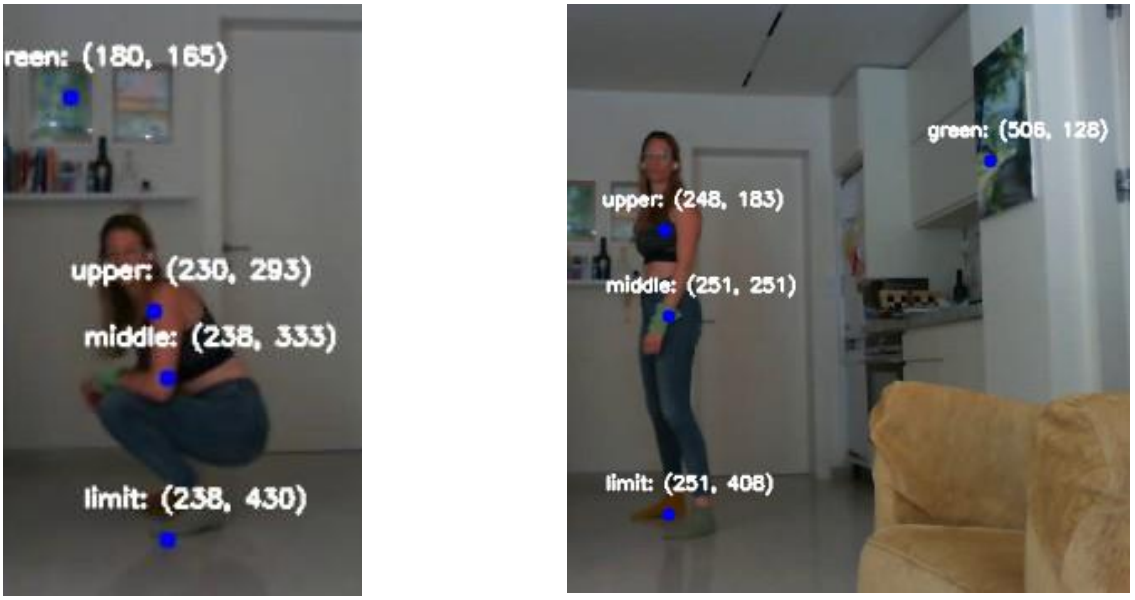


Figure 12-false identification of green

To enhance accuracy, we refined the area of interest to solely encompass the region defined by the binary mask. We utilized a less filtered version of the binary mask, prior to the application of the final two median filters, to minimize false negatives resulting from the reduced region of interest. This approach ensured that environmental objects did not interfere with the detection of red and green accessories. The improved results are below:



Figure 13-correct identification of green

Analysis

In this section we will analyze our lean detection algorithm. To

detect a lean, the next steps need to be achieved:

1. Getting the background for background subtraction
2. Creating the binary mask
3. Calculating the center of mass, and consecutively the upper/lower center of masses.
4. If a big enough tilt was created between the upper and lower center of mass, we conclude that a lean was performed.

As we can see, for each step there are a few parameters that will affect their performance.

In our analysis we will examine how different kinds of noises at different strengths affect the performance of the lean detection algorithm.
We will use two different kinds of noises: Salt & Pepper and Gaussian.

Lean Detection

We will examine our lean detection performance using confusion matrix and calculate our learned performance metrics using F1 scores.

- Confusion Matrix - A table that counts the True/False Positive/Negative prediction made by the model.
- F1 Score - A metric that combines Precision(accuracy of positive predictions made by the model) and recall(proportion of actual positives that were correctly identified by the model), this score penalizes extreme values more than the arithmetic mean, making the F1 score a better representation of the balance between precision and recall. It ranges from 0 to 1, where 1 indicates perfect precision and recall, and 0 indicates either no precision or no recall.

First, we will collect the data from the sessions and create the confusion matrix. Each row represents the true actions performed by the player during a gameplay session, while each column represents the actions detected by the system.

We will start by showing our confusion matrix for the sessions without any additional noise and its corresponding performance metrics:

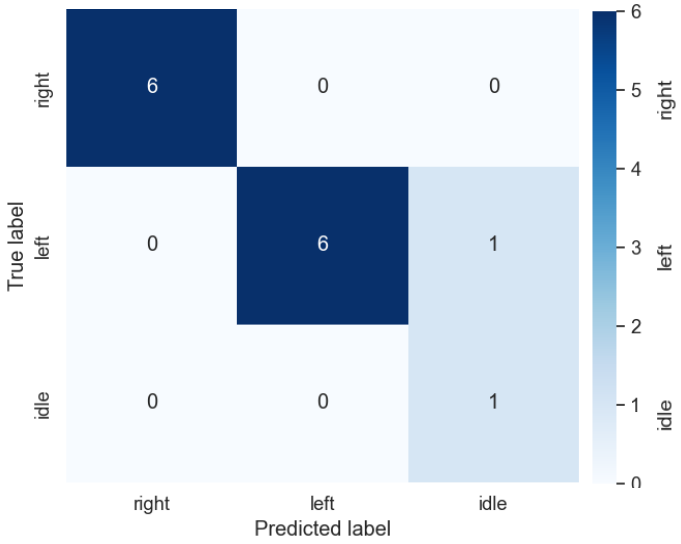


Table 1-confusion matrix, no additional noise

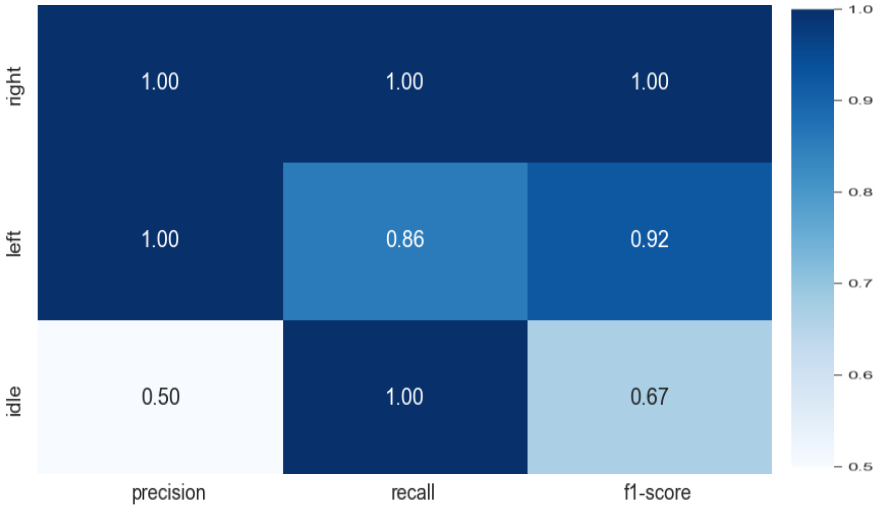


Table 2-F1 score, no additional noise

As we can see, when we don't add any additional noise, our performance metrics are almost optimal for the gameplay sessions.

Now let's analyze the results for an additional noise.

First, we will look at an addition of Salt & Pepper noise.

As for the parameter of noise, we introduced a percentage parameter that will represent the probability for the pixel's value to be inverted.

Salt & Pepper noise with 1% probability for a pixel's value to be inverted:

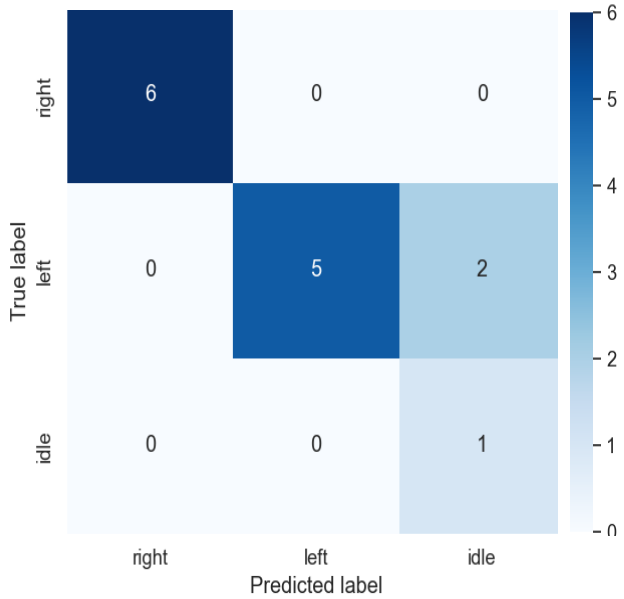


Table 3-confusion matrix, salt & pepper noise, 1%

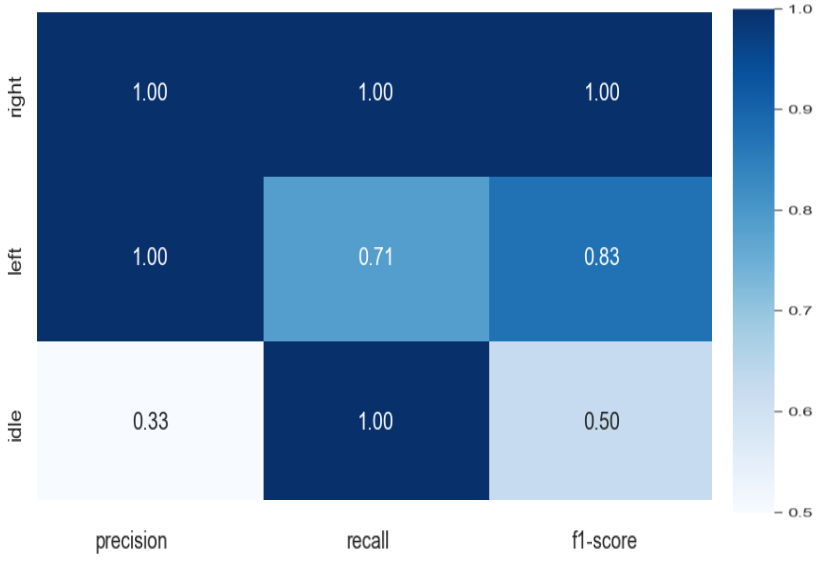


Table 4-F1 score, salt & pepper noise, 1%

Salt & Pepper noise with 2% probability for a pixel's value to be inverted:

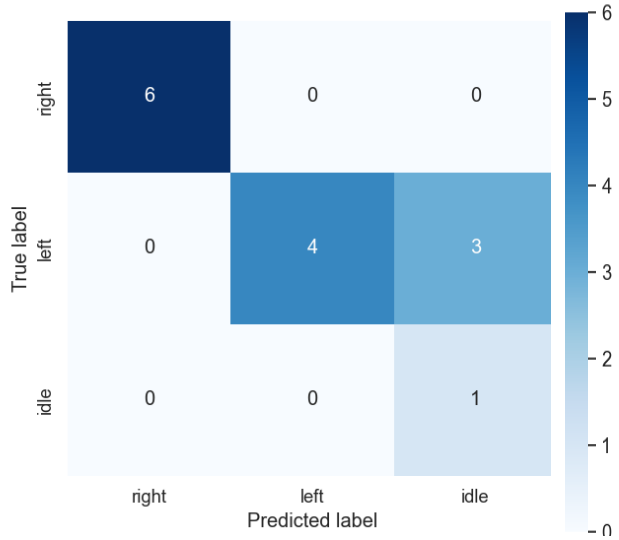


Table 5-confusion matrix, salt & pepper noise, 2%

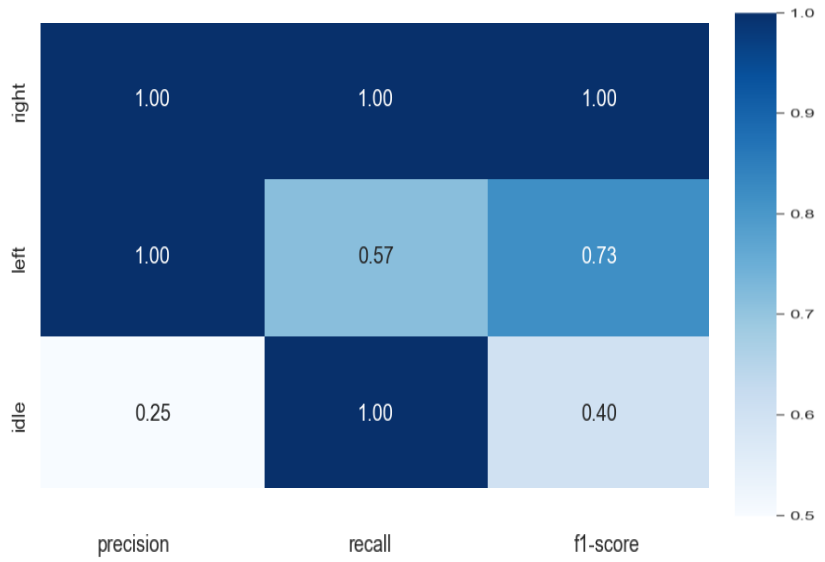


Table 6-F1 score, salt & pepper noise, 2%

Salt & Pepper noise with 3% probability for a pixel's value to be inverted:

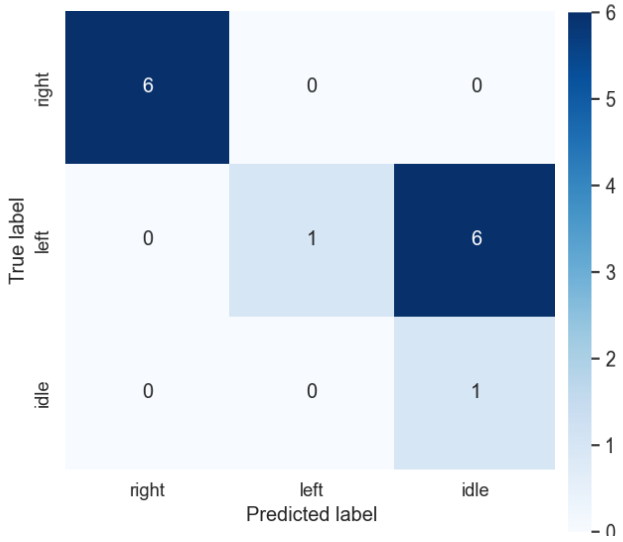


Table 7-confusion matrix, salt & pepper noise, 3%

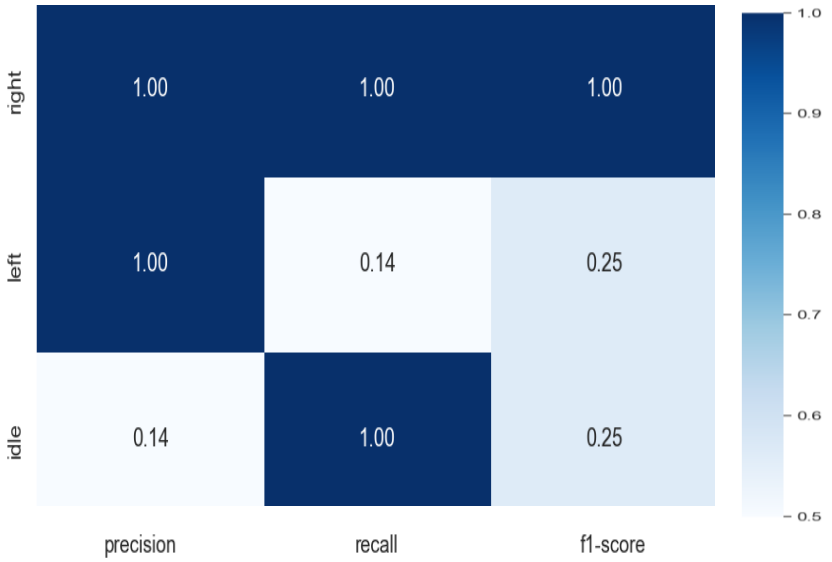


Table 8-F1 score, salt & pepper noise, 3%

The presence of incremental noise adversely affects our model's performance, as anticipated. With each increment, the noise alters the position of our center of mass, consequently influencing the activation criteria for lean detection. Notably, our model exhibits greater resilience to noise in detecting right leans compared to left lean.

Next, lets see how does an additional gaussian noise with unit variance with different amplitude affect our model.

Gaussian noise with 4/255 amplitude:

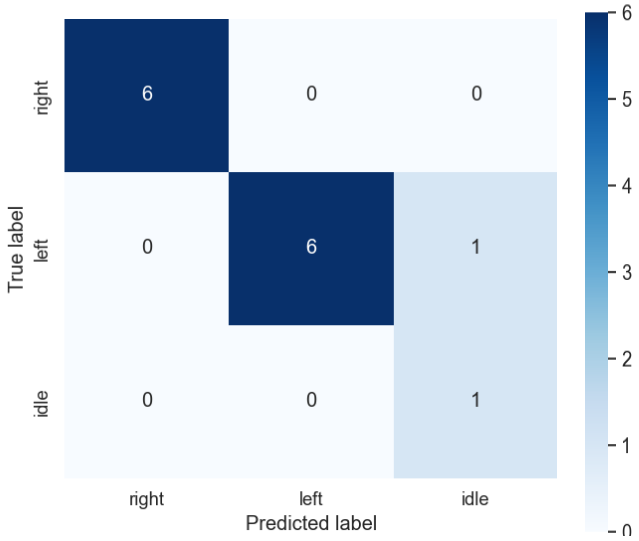


Table 9-confusion matrix, gaussian noise, 4/255

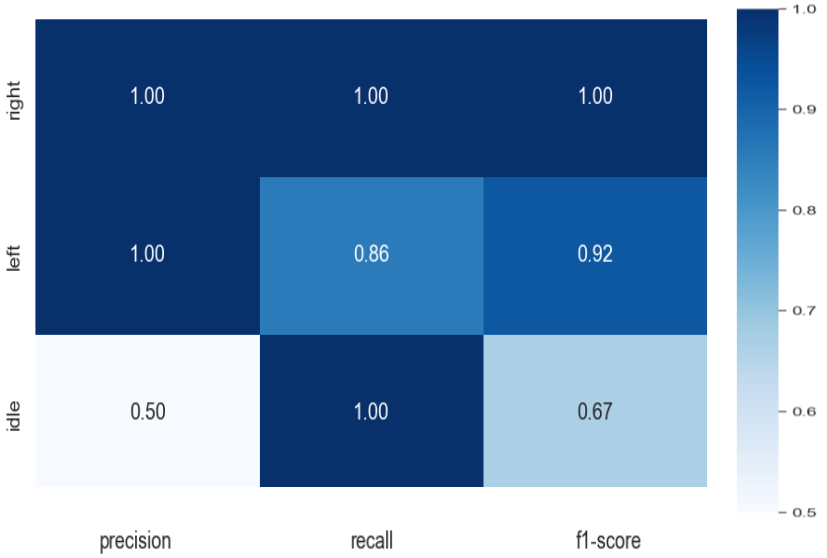


Table 10-F1 score, gaussian noise, 4/255

Gaussian noise with 12/255 amplitude:

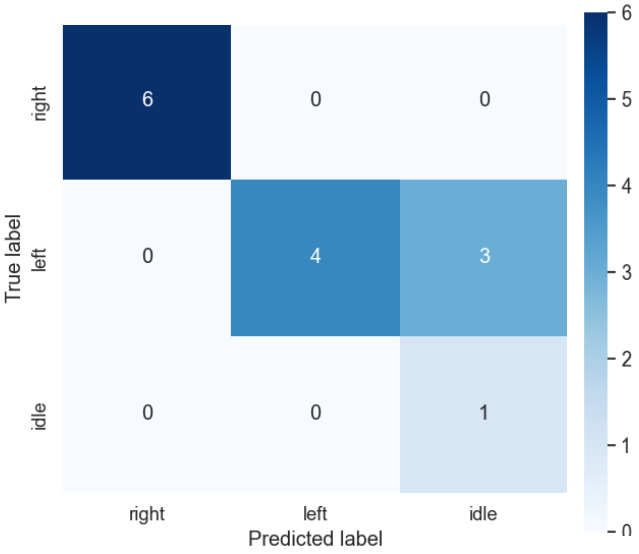


Table 11-confusion matrix, gaussian noise, 12/255

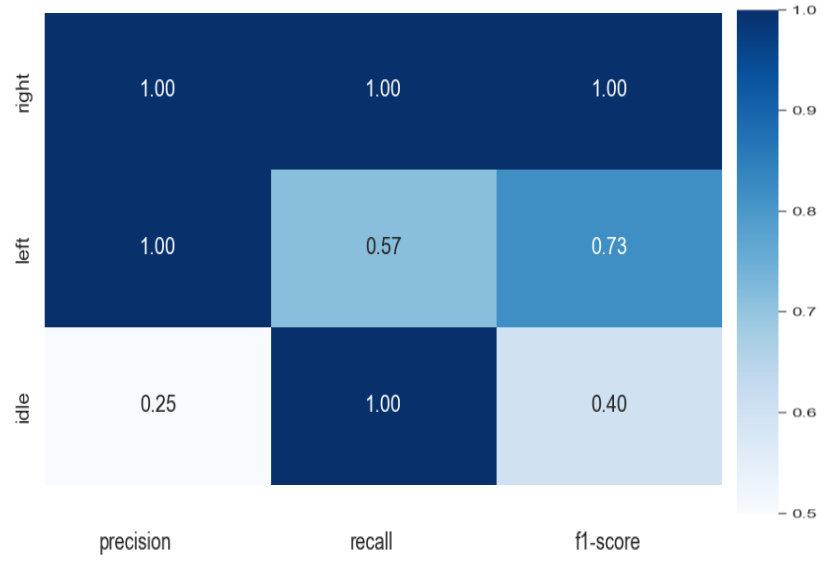


Table 12-F1 score, gaussian noise, 12/255

Gaussian noise with 16/255 amplitude:

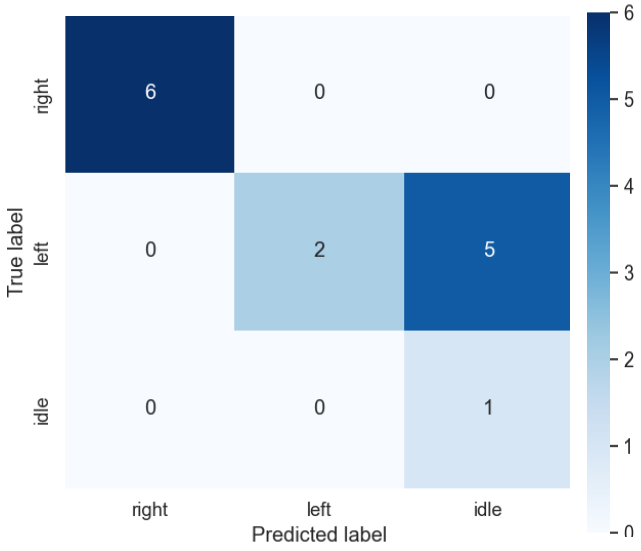


Table 13-confusion matrix, gaussian noise, 16/255

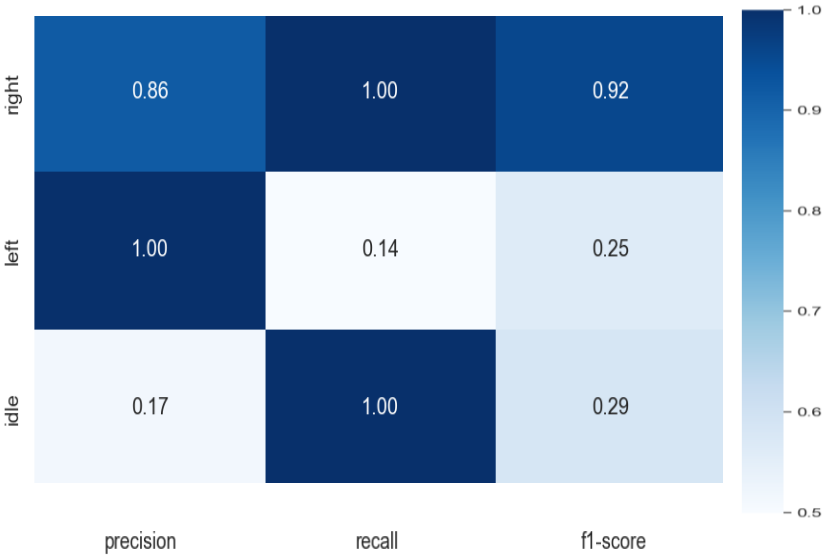
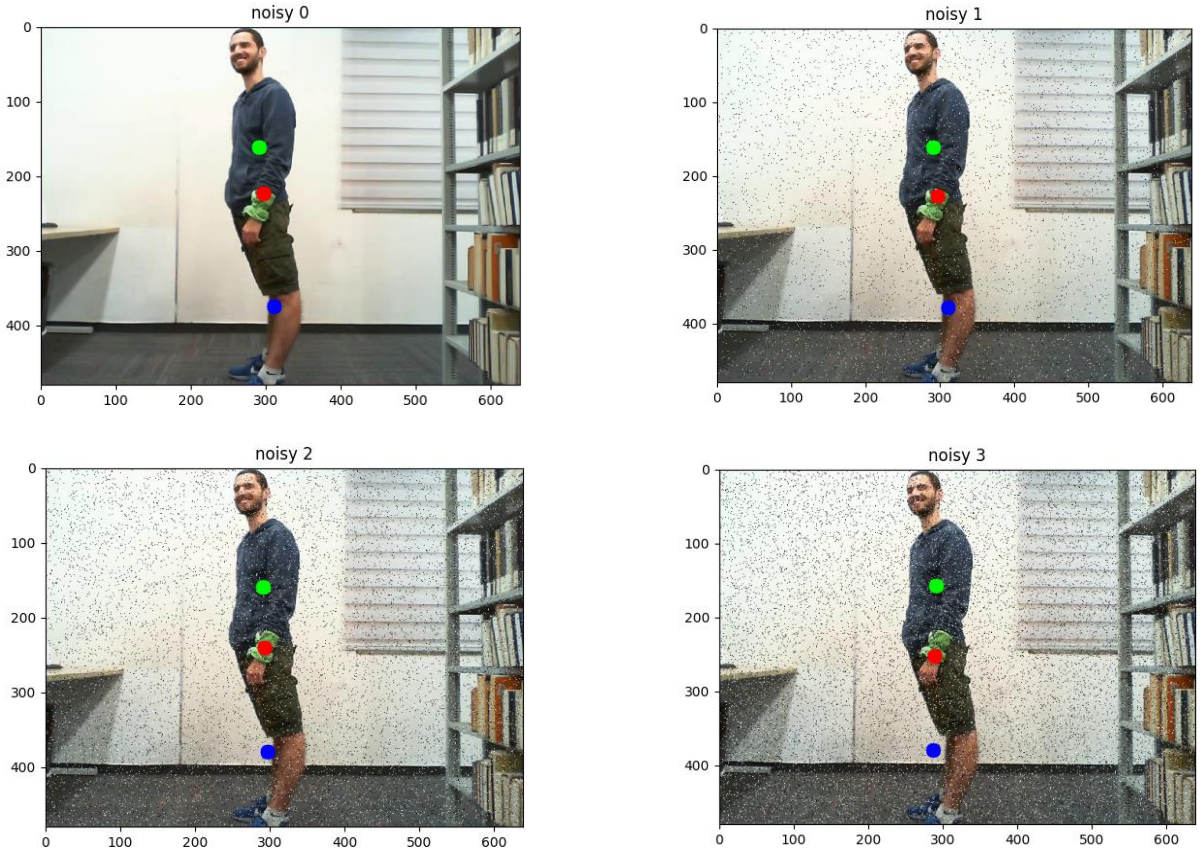


Table 14-F1 score, gaussian noise, 16/255

Similar to the impact of Salt & Pepper noise, we observe a degradation in performance as the noise intensity increases. Just as with Salt & Pepper noise, our model exhibits greater resistance to detecting right leans.

Salt &Pepper noisy images:



Conclusion

Our analysis of the lean detection algorithm reveals its resilience to noise, albeit with varying degrees of effectiveness. As anticipated, the performance of our model deteriorates with increasing noise levels. Interestingly, we observe that our model exhibits greater resistance to Salt & Pepper noise compared to Gaussian noise. Furthermore, the ability to accurately detect right leans appears to be more robust in the presence of noise, whereas the detection of left leans and idle states is less reliable. This is partly due to the positioning within the library our project occupied; we opted not to restrict our background beyond ensuring its mostly static nature. Consequently, significant noise had a more pronounced effect compared to scenarios where the background maintained uniform coloration.

These findings underscore the importance of continued refinement and optimization of our algorithm to enhance its performance across various noise conditions. Overall, the performance of our algorithms was good in the presence of the typical noise, and we saw that even in the presence of artificial noise of different degrees, we still had good results showing our model robustness.

Summary

The project aims to deliver an immersive snowboarding experience with a focus on user satisfaction. Beginning with setting up a static camera to capture the game environment, we isolate the player's presence through background scanning and binary mask creation, enabling real-time analysis and interaction. Utilizing Digital Image Processing and Computer Vision algorithms like Canny edge detection, Hough Transform, and HSV color space analysis, we ensure precise interpretation of player movements. Through meticulous implementation of thresholds, calibration parameters, and tailored algorithms, we strike a balance between accuracy and efficiency, resulting in smooth gameplay. Our approach includes algorithms for player position detection, fast, slow, jump, squat recognition, leaning analysis, and grab detection, enhancing gameplay dynamics. By accurately capturing and interpreting various player actions in real-time, we enable intuitive and responsive interactions, fostering a deeper sense of engagement with the game.

Another exciting aspect not discussed during the process and during this report is the concept of "combos" in Snowboard Party. Snowboard Party offers a range of complex maneuvers achieved by chaining together specific actions and tricks. While our focus was on accurately recognizing basic moves through image processing, executing the right combinations of moves unlocks all available combos for the player. However, mastering these combos may pose a challenge for inexperienced players, such as performing tricks while in mid-air.

Collaboration was at the heart of our endeavor, as we embraced teamwork and shared knowledge, each member contributing unique insights and skills to the project. This journey was marked by countless hours of learning, coding, and debugging, yet amidst the challenges, we discovered the true extent of our capabilities.

Our project clearly surpassed our initial expectations, reflecting our dedication and enthusiasm for learning new things and creating an immersive snowboarding experience. We are more than happy with the results, and proud of the journey we've taken.

Bibliography

[1] openCV - Documentation Retrieved from https://docs.opencv.org/4.x/

[2] Riklin , T. R. (n.d.). Introduction to Digital Image Processing.

Retrieved from https://www.ee.bgu.ac.il/~rrtammy/DIP/DIP

[3] Snowboard Party Lite - Retrieved from

https://snowboard-party-lite.en.softonic.com/

[4] Forsyth, David A., and Jean Ponce. "A modern approach." Computer vision: a modern approach (2003).

Links

Link to a drive with the updated **video** for this project (access for all @post users)

Video - Snowboard Party

Link to the GitHub repository of this project –

GitHub Repo Snowboard Party